

Laboratory 6: WiFi Client

Objectives

After completing this lab, students will be able to:

1. Connect the M5Stack to a **WiFi** network.
2. To understand the basics of an **API (Application Programming Interface)** and **HTTP GET** requests.
3. To request data from a public API and view the full response in the Serial Monitor.
4. To parse a **JSON** response to extract and display specific, useful information on the LCD.

Key Concepts

WiFi & Internet Connection

The M5Stack contains an ESP32 chip with built-in WiFi. It can connect to any WiFi network and then communicate with servers worldwide. You will need to include `<WiFi.h>` and `<HTTPClient.h>`, which are built-in for the M5Stack's ESP32 board. No installation is required.

- `WiFi.begin(ssid, password)` - Tells the ESP32 to start connecting to a WiFi network
 - `ssid`: The network name (e.g., "MyHomeWiFi" or your phone's hotspot)
 - `password`: The network password
 - Important: Connection takes time and may fail if credentials are wrong
- `WiFi.status()` - Returns the current connection state
 - `WL_CONNECTED` = successfully connected
 - Other values = still connecting or connection failed
- `WiFi.localIP()` - Returns your M5Stack's unique IP address on the network (e.g., `192.168.1.100`) - Like a mailbox address for receiving data

The HTTP Protocol and REST APIs

Your M5Stack can connect to the internet just like your phone or laptop. Once connected, it can request data from servers around the world. A common way to get data is by using a **public API**.

- **API (Application Programming Interface)**: Think of an API as a restaurant menu for programs. It's a defined set of requests that a program can make to a server to get specific information (e.g., "get the current weather," "get a random fact").
- **Common HTTP requests**:

- **GET** - The HTTP **GET** method requests data from a specified resource, which is indicated by a URL. It's like asking the server, *"Can you get me the data at this specific web address (URL)?"*
- **POST** - The HTTP **POST** method sends data to be processed on a specified resource on the web server.

In this lab, we will focus on the **GET** method of the HTTP protocol.

Example code to make an API request:

```
HTTPClient http;           // (1) Create an HTTPClient object
String payload = "";       //      store result from request
http.begin(url);           // (2) Prepare to make request to API at url
int httpCode = http.GET();  // (3) Send request to API
if (httpCode > 0) {         //      A code > 0 means we got a response
    payload = http.getString(); // (4) Get the response text (the JSON)
} else {
    // request failed, show error
    Serial.printf("Request to %s failed, error: %s\n",
                  url.c_str(), http.errorToString(httpCode).c_str());
}
http.end();                // (5) Important: free up resources
```

Some common HTTP code results:

200: OK
400: Bad Request
401: Unauthorized
403: Forbidden
404: Not Found
500: Internal Server Error
503: Service Unavailable

JSON (JavaScript Object Notation): When the server responds to an API request, it usually sends the data back in **JSON** format. **JSON** is a simple text format using **key-value** pairs, for example:

```
{"status": "success", "city": "Bangkok", "country": "Thailand"}
```

Or show in Pretty-print (easier to read):

```
{
  "status": "success",
  "city": "Bangkok",
  "country": "Thailand"
}
```

String Objects & Methods

This lab's main challenge involves processing text received from the internet. We will use the Arduino **String** object, which has helpful built-in functions called methods. The two most important for this lab are `.indexOf()` and `.substring()`.

- a) **.indexOf("search_text")** – Finds the starting position (index) of text within a String.

Return: An integer (the position), or -1 if not found. (The first character is at index 0)

Example:

```
String myText = "Hello, world!";
int position = myText.indexOf("world"); // position will be 7,
// because 'w' is the 7th character (starting from 0).
```

- b). **.substring(startIndex, endIndex)** - Extracts a portion of a String.

Return: A new String with characters from `startIndex` up to (but NOT including) `endIndex`

startIndex: The index where you want to start copying.

endIndex (Optional): The index before which to stop copying. If you leave this out, it copies to the end of the string.

Example:

```
String myText = "Hello, world!";
String greeting = myText.substring(0, 5);
// greeting will be "Hello"
// (it copies from index 0 up to, but not including index 5).
String greeting2 = myText.substring(7);
// greeting2 -> "world!"
```

Putting It Together: A Mini-Challenge

Imagine you have the following JSON String from an API and you want to extract the city name "Bangkok".

```
{"status":"success","city":"Bangkok","country":"Thailand"}
```

Goal: Extract the value "Bangkok" for the "city" key.

```
String jsonData =
"{\"status\": \"success\", \"city\": \"Bangkok\", \"country\": \"Thailand\"}";

// STEP 1. Define the pattern that comes right before our data.
String startPattern = "\"city\": \"";

// STEP 2. Find where the pattern begins
int startIndex = jsonData.indexOf(startPattern)
// jsonData.indexOf(startPattern) finds where "city":" starts.
// We add its length to get the index right after it.
startIndex += startPattern.length();

// STEP3. Find where the pattern ends (the next quote character).
String endPattern = "\"";
int endIndex = jsonData.indexOf(endPattern, startIndex);

// 4. Extract the substring!
String city = jsonData.substring(startIndex, endIndex);

// city will now contain the String "Bangkok"
```

This is the exact technique you will use in the lab challenge.

Lab Exercises

Task 1: Connecting to WiFi Network (lab_6_1)

Objective: Connect your M5Stack to a WiFi network and display the IP address

- 1.1 Connect your M5Stack to your laptop and open Arduino IDE.
- 1.2 Create a new sketch, copy the provide code, and save as **lab_6_1**

```
#include <M5Unified.h>
#include <WiFi.h>

// (1) --- WiFi Credentials ---
// TODO: Replace these with your actual WiFi network name and password
#define WIFI_SSID "YourWiFiName"
#define WIFI_PASSWORD "YourWiFiPassword"

// --- Best Practice: Use constants for configuration ---
const long connectionTimeout = 15000; // Timeout 15 seconds
const int statusCheckDelay = 500;    // Delay between checks

void setup() {
  M5.begin();
  Serial.begin(115200);
  M5.Display.setTextSize(2);
  M5.Display.println("Lab 6 - WiFi Client");
  startWiFi();
}

void loop() {
  // Nothing to do here for this part
}

void startWiFi() {
  M5.Display.printf("Connecting to %s ", WIFI_SSID);
  Serial.printf("Connecting to %s ", WIFI_SSID);

  // (2) Start the WiFi connection
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  // Remember the time when we started trying to connect
  unsigned long startTime = millis();

  // (3) The loop will now stop if we connect OR if the timeout is reached
  while (WiFi.status() != WL_CONNECTED
    && (millis() - startTime) < connectionTimeout) {
    delay(statusCheckDelay); // Delay between checks
    M5.Display.print(".");
    Serial.print(".");
  }

  // After the loop, check WHY we exited.
  if (WiFi.status() == WL_CONNECTED) {
    // We connected successfully!
    M5.Display.setTextColor(TFT_GREEN);
    M5.Display.println("\nWiFi Connected!");
    M5.Display.print("IP: ");
    M5.Display.println(WiFi.localIP());
    M5.Display.setTextColor(TFT_WHITE);

    Serial.println("\nWiFi Connected!");
    Serial.printf("IP Address: ");
    Serial.println(WiFi.localIP());
  }
}
```

```

} else {
  // The timeout was reached. We did not connect.
  M5.Display.setTextColor(TFT_RED);
  M5.Display.println("\nWiFi Connection Failed!");
  M5.Display.println("Please check credentials or network and reset.");
  M5.Display.setTextColor(TFT_WHITE);
  Serial.println("\nWiFi connection failed after 15 seconds.");
}
}

```

Explanation:

- **WiFi.begin(WIFI_SSID, WIFI_PASSWORD);**
This is the key command. It tells the M5Stack's ESP32 chip to start the process of connecting to the network using the credentials you provided.
- **while (WiFi.status() != WL_CONNECTED)**
This is a very important loop. It translates to: "While the WiFi status is not equal to 'connected'..."
- **WiFi.status()** checks the current connection state.
- **WL_CONNECTED** is a constant that represents a successful connection.
- **WiFi.localIP()** is the unique IP address the M5Stack received from your WiFi router.

1.3 If you see the messages "WiFi Connected!" and IP address, the mean you are done with **Task 1**.

Troubleshooting & Best Practices

- **Check for Typos:** The most common reason for a failed connection is a typo in the **WIFI_SSID** or **WIFI_PASSWORD**. They are case-sensitive!
- **Use Simple SSIDs:** Avoid emojis or special characters.
- If using **iPhone** hotspot: Enable "Maximize Compatibility" in Personal Hotspot setting (see picture).



Task 2: Connecting to Public API Service – A Random Joke

Objective: Make your first API request and display the full response

Let's start with a fun and simple API that gives us a random joke. This helps us practice making a request and seeing the full response without complex parsing.

URL Endpoint: `http://official-joke-api.appspot.com/random_joke`

2.1 Use code provided in myCourseVille. Save as **lab_6_2**

2.2 **TODO:** Add `#include <HTTPClient.h>` to the include section. This library provides API request methods.

2.3 The `loop()`, `handleButtonAPressed()`, and `performApiRequest()` functions:

```

void loop() {
    M5.update();
    if (M5.BtnA.wasPressed()) {
        Serial.println("Button A is pressed");
        handleButtonAPressed();
        Serial.println("-----");
    }
    delay(200); // small delay to deal with slower network access
}

void handleButtonAPressed() {
    String apiURL = "http://official-joke-api.appspot.com/random_joke";
    String payload = performApiRequest(apiURL);
    if (payload != "") { // get result from request
        // TODO: Count successful responses and show the count on both Serial Monitor
        and LCD

        // --- Display on Serial Monitor ---
        // TODO: modify printf() to show the count
        Serial.printf("--- RAW RESPONSE --- \n");
        Serial.println(payload);

        // --- Display on LCD ---
        // TODO: modify printf() to show the count
        M5.Display.clear();
        M5.Display.setCursor(0, 0);
        M5.Display.printf("Got a new joke! \n");
        M5.Display.println("Check Serial Monitor for the full text.");
    } else {
        M5.Display.setTextColor(TFT_RED);
        M5.Display.println("Request failed.");
        M5.Display.setTextColor(TFT_WHITE);
    }
}

String performApiRequest(String url) {
    Serial.print("Making request to: ");
    Serial.println(url);

    HTTPClient http;                // (1) Create an HTTPClient object
    String payload = "";            // store result from request
    http.begin(url);                // (2) Prepare to make request to API at url
    int httpCode = http.GET();      // (3) Send request to API
    if (httpCode > 0) {              // A code > 0 means we got a response
        payload = http.getString(); // (4) Get the response text (the JSON)
    } else {
        // request failed, show error
        Serial.printf("Request to %s failed, error: %s\n",
                      url.c_str(), http.errorToString(httpCode).c_str());
    }
    http.end();                     // (5) Important: free up resources
    return payload;
}

```

2.4 **Testing** - Upload and open Serial Monitor. **Press Button A** – You should see the joke JSON on Serial Monitor + "Got a new Joke!" on LCD.

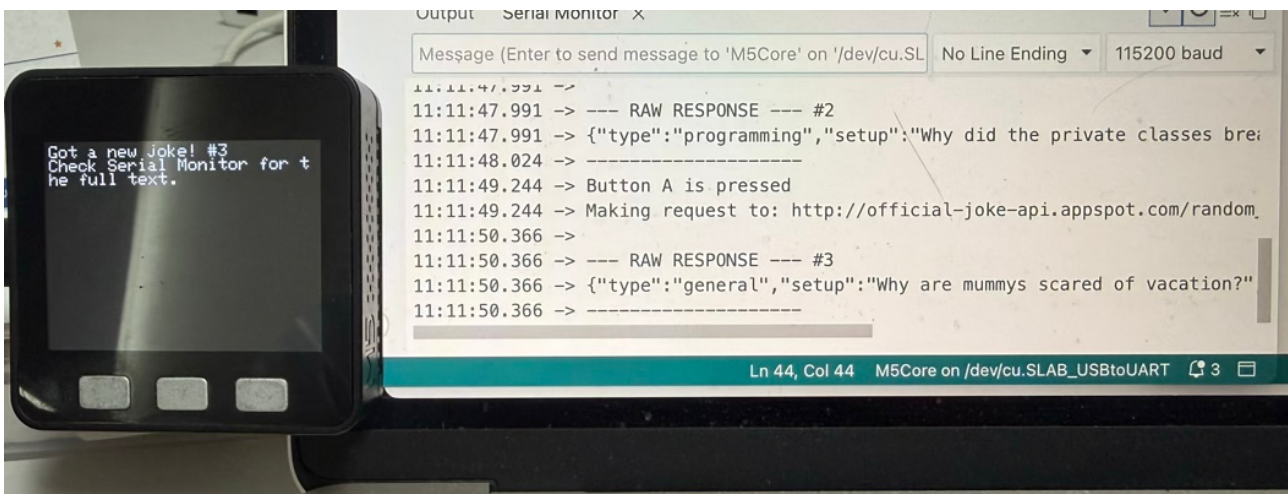
2.5 **IMPORTANT:** If you see, "Got a new Joke!" on LCD immediately after successfully connected to WiFi without pressing Button A. The program does not work as expected. Uncomment the line `delay(200)`. This is because the `http.GET()` command used in API calls is a "**blocking**" function. It can pause your `loop()` for a second or more while it

waits for the server to respond. This long, unpredictable pause can sometimes confuse the `M5.update()` function on the next loop cycle, making it incorrectly think a button is still being pressed. This would cause unwanted, repeated API calls.

2.6 **TODO:** Add a global variable: `responseCount` to track successful response and increments each time you get a successful response from the API (**See Lab 5 as guideline**).

2.7 **TODO:** Modify the code to show the successful response number (`k`) to both the LCD and Serial Monitor.

- LCD: Got a new joke! `#k`
 - Serial Monitor: --- RAW RESPONSE `#k` ---



Task 3: Challenge – Multiple APIs & JSON String Parsing

Objective: Use buttons to fetch different data from different APIs and parse JSON responses.

Now, let's combine everything into one final program. Create a new sketch (**lab_6_3**) that uses code skeleton from **myCourseville** and adds:

- **Button A:** Fetch a new random joke and extract **"setup"** and **"punchline"** sentences from the JSON result ([see mini challenge](#)). Response example:

```
{"type": "programming", "setup": "There are 10 types of people in this world...", "punchline": "Those who understand binary and those who don't", "id": 28}
```

LCD: Displays the **"setup"** sentence in **ORANGE** and the **"punchline"** sentence in **GREEN**, each on separate lines.

- **Button B:** Fetch the current **Bitcoin** price in US dollars from `"https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT"`
 - **Serial Monitor:** Display the full JSON response.
 - **LCD:** Display only the **Bitcoin** price (e.g., 88302.01000000).

Advanced Topics – Security & Best Practices

Problem #1: Hardcoding Secrets. In this lab, WiFi credentials are written directly in the sketch.

This is fine for learning, but DANGEROUS for real projects: - If you upload this code to GitHub, the whole world sees your WiFi password – Anyone with access to the code can connect to your network. For example:

```
#define WIFI_SSID "MyHomeWiFi";  
#define WIFI_PASSWORD "myPassword123";
```

Solution: Use a Credentials File.

1. In Arduino IDE, create a new TAB named “`credentials.h`”
2. In this file, add

```
#define WIFI_SSID "MyHomeWiFi"  
#define WIFI_PASSWORD "myPassword123"
```

3. In your main sketch, include this file and remove the defines:

```
#include "credentials.h"  
#define WIFI_SSID "MyHomeWiFi"  
#define WIFI_PASSWORD "myPassword123"
```

4. Add `credentials.h` to your `.gitignore` file (tells Git not to upload it)

Benefits:

- Credentials stay out of the main sketch
- Easy to change credentials without editing main code
- Can share code on GitHub safely

Problem #2: Requesting to unstable API services or unstable network connectivity.

In this lab, we connect to stable API services, which response immediately. In real world, some API services might be busy during your request. As a result, you might get "Server not response".

Solution: Implement more robust API requests, by adding retry feature to `performApiRequest()` in case of network instability. Replace `performApiRequest()` with the new version provided.

```
String performApiRequest(String url) {  
    HTTPClient http;  
    for (int i = 0; i < 5; ++i) { // loop to retry  
        http.begin(url);  
        int httpCode = http.GET();
```



```
    if (httpCode > 0) {
        ++responseCount;
        String payload = http.getString();
        http.end();
        return payload;
    } else {
        Serial.printf("Request to %s failed, error: %s\n",
                      url.c_str(), http.errorToString(httpCode).c_str());
        Serial.printf("Retry...#%d\n", i+1);
    }
    delay(3000); // wait before retrying
}
Serial.println("Server not response.");

http.end();
return "";
}
```

For a request, we retry 5 times. There must be `delay()` between each retry. Otherwise, from the API server's point of view, this rapid-fire succession of requests looks like a **denial-of-service (DoS) attack**. Many automated systems will detect this aggressive behavior and temporarily (or permanently) **block your IP address** to protect themselves.

Lab 6: WiFi Client

Section: _____ Date: _____

Members

Name: _____ Student ID: _____

Name: _____ Student ID: _____

Name: _____ Student ID: _____

Task	Graded by
1. Connecting to WiFi Network	
2. Connecting to API: Button A fetches new request, show successful request # LCD: Got a new joke! #k Serial Monitor: --- RAW RESPONSE #k ---	
3. API Client: Button A – display sentences Button B – display Bitcoin price	

4. Reflection about this lab. See assignment in **MycourseVille**

- Q1.** In this lab, you fetched data from two public APIs: jokes and Bitcoin price. Describe one real-world scenario where an M5Stack device would need to fetch live data from the internet. What information would it display, and why would it be useful?
- Q2.** In Task 3, you extracted data from JSON by using `.indexOf()` and `.substring()` to find and pull-out specific text. Why is it important for programmers to understand string processing, even though JSON libraries (like `ArduinoJson`) exist and can do this automatically? Describe a situation where manual parsing might be useful or necessary.
- Q3.** In the Advanced Topic, you learned about retry logic: if an API call fails, the code waits a few seconds and then tries again (up to a maximum number of attempts). Why is the `delay(3000)` between retries important? What could go wrong if the code retried immediately and repeatedly without any delay?
- Q4.** In this lab, Task 1 auto-connects to WiFi automatically when powered on. In a real IoT device (like a smart thermostat or weather station), when would auto-connect be better? When would you prefer manual connection control instead? Give one example scenario for each.